Optimización de MAXIM con expertos: resumen

Oct 28, 2026 · @Mati

Objetivo y resultado principal

Reemplazamos un bloque pesado de MAXIM por expertos pequeños, uno por tarea. El modelo bajó **13,5 % sus parámetros frente al baseline, de 14,18 M a 12,26 M**, sin perder calidad en test. Un módulo pequeño común, con el mismo costo, sí pierde calidad.

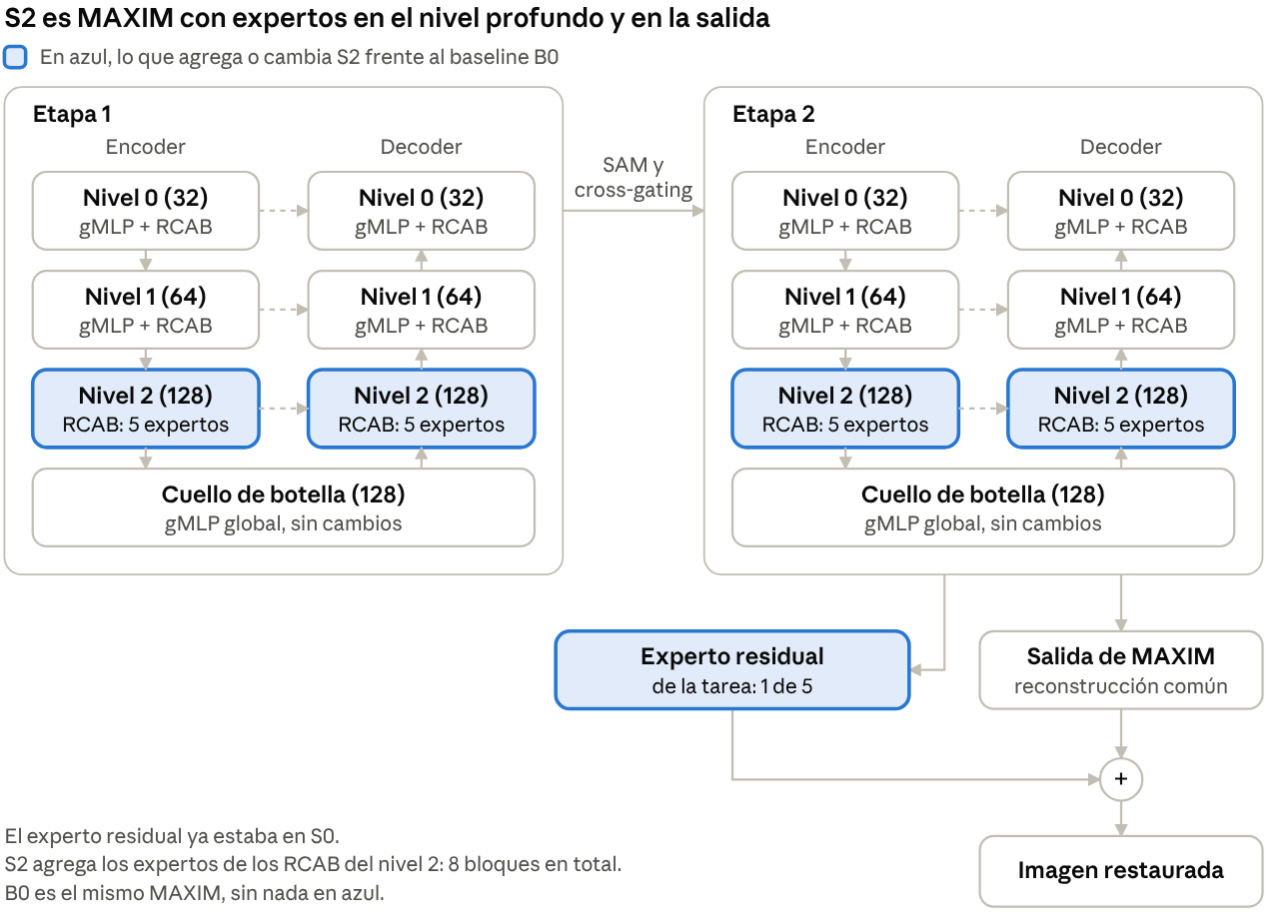
El objetivo de esta fase era reducir el tamaño y el costo de MAXIM aprovechando los expertos por tarea. Partimos de dos modelos ya entrenados, MAXIM S-2 con cinco tareas: deblur, dehaze, denoise, derain y enhance.

- **Baseline (B0):** MAXIM sin expertos.
- **Shared (S0):** el mismo MAXIM, cuya reconstrucción es común a todas las tareas, más cinco expertos residuales chicos, uno por tarea. Cada experto suma una corrección a esa reconstrucción. La tarea de cada imagen se conoce de antemano y se pasa al modelo: no hay router que la decida.

En el test del 4 de octubre, shared y baseline daban prácticamente la misma calidad, con +0,009 dB de PSNR macro. Por eso la fase se orientó a costo y no a mejorar calidad.

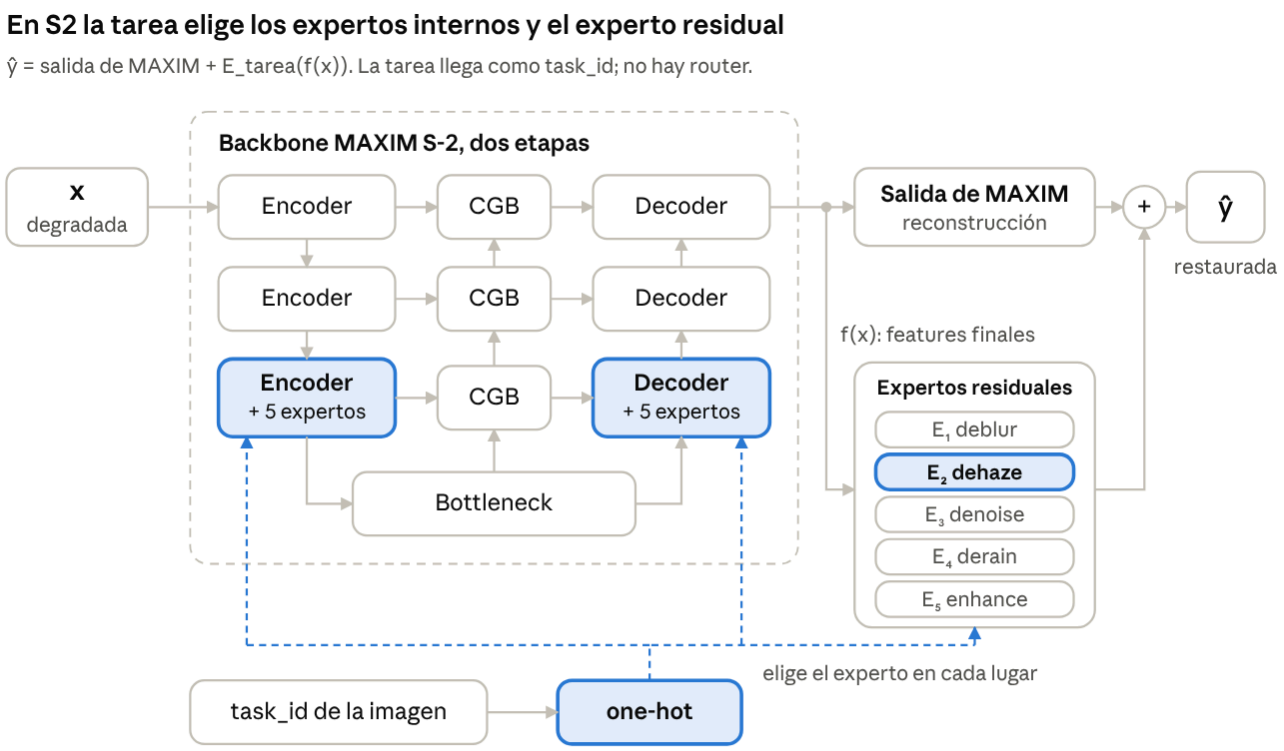
Qué reemplazamos

Reemplazamos las convoluciones de los ocho bloques RCAB del nivel más profundo de la red, con 128 canales: **2,36 M parámetros, el 16,7 % del baseline**. Lo elegimos con el benchmark de costo: es una parte grande del modelo y su reemplazo no cambia la estructura de MAXIM.



MAXIM S-2, vista general · B0 frente a S2

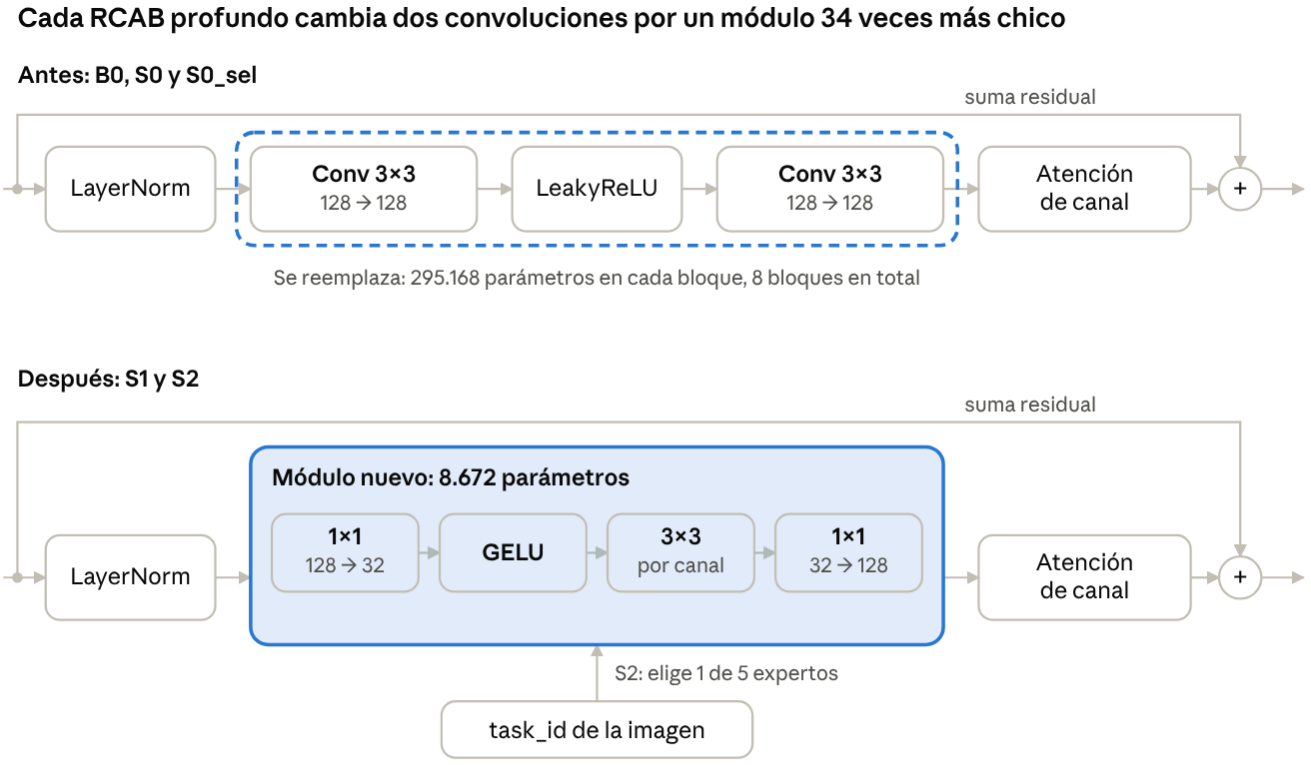
MAXIM S-2 tiene dos etapas. Cada una es una U-Net de tres niveles, de 32, 64 y 128 canales, con un cuello de botella global. Los expertos de S2 están en el nivel más profundo de las dos etapas y en la salida final.



Arquitectura de S2 · versión actualizada de la figura MAXIM+MoE de la tesis

A diferencia de la figura MAXIM+MoE original, no hay router: la etiqueta de la tarea elige el experto en cada lugar. El experto residual suma una corrección a la salida de MAXIM, en lugar de reconstruir la imagen desde x.

- **Antes:** en cada uno de los ocho lugares, dos convoluciones 3×3 de 128 a 128 canales, con 295.168 parámetros.
- **Después:** un módulo pequeño con cuello de botella. Proyecta de 128 a 32 canales, aplica GELU y una convolución 3×3 por canal, y vuelve a 128 canales. Tiene 8.672 parámetros, 34 veces menos.
- **Módulo común o expertos.** El módulo nuevo puede ser uno solo, compartido por las cinco tareas. También puede ser un banco de cinco expertos, uno por tarea. En el banco se ejecuta sólo el experto de la tarea de cada imagen: los otros cuatro no se calculan.



S1 usa un solo módulo común a las cinco tareas. S2 tiene cinco copias, una por tarea, y ejecuta sólo la de la imagen.

Bloque RCAB del nivel profundo, antes y después · maxim/models/maxim.py y compression.py

La normalización, la atención de canal y la suma residual del RCAB no cambian. Sólo cambia lo que está entre ellas.

El resto de la red se mantuvo igual y congelado, incluidos los gMLP multi-eje, el cross-gating y los expertos residuales finales del shared.

Versiones comparadas

La comparación central es S2 contra S1. Tienen el mismo costo de ejecución y sólo difieren en si el módulo nuevo es común o especializado por tarea.

Versión	Qué es	Parámetros
B0	Baseline, MAXIM sin expertos	14,18 M
S0	Shared original: calcula las cinco correcciones y se queda con la del experto de la tarea	14,27 M
S0_sel	Shared con ejecución selectiva: calcula sólo el experto de la tarea. Mismos pesos y misma salida que S0	14,27 M

Versión	Qué es	Parámetros
S1_d32	S0_sel con un módulo común de ancho 32 en lugar de las convoluciones	11,98 M
S2_d32	**S0_sel con cinco expertos de ancho 32, uno por tarea**	**12,26 M**

En el benchmark de costo también medimos otras versiones como referencia:

- S1 y S2 con ancho 64.
- S2 calculando los cinco expertos, para ver cuánto cuesta no seleccionar.
- Tres límites teóricos: sin las convoluciones reemplazadas, sin cross-gating y sin gMLP multi-eje.
- B0 y S2 en bf16, y B0 con rematerialización para ahorrar memoria de entrenamiento.

Pruebas realizadas

Hicimos dos benchmarks de costo, dos adaptaciones con semillas distintas, la validación de cada una y un test final único. Todas las corridas fueron en Colab con una GPU L4.

Fecha	Prueba	Qué se obtuvo
28/10	Validación con las dos semillas	S2 supera a S1 por 0,115 dB promediando las semillas, con el mismo patrón por tarea en las dos. La semilla 1 reproduce la validación del 21/10
28/10	Test final, 4.730 imágenes	S2 supera a S1 en las dos semillas y empata con el shared. Un primer intento, el 27/10, se cortó por falta de memoria antes de evaluar ningún modelo
23–25/10	Adaptación de S1 y S2, semilla 2	Mismo resultado que la semilla 1: S2 queda 0,118 dB por encima de S1 en validación
21/10	Validación de la semilla 1, 2.303 imágenes	S2 supera a S1 por 0,112 dB. bf16 pierde entre 0,67 y 0,90 dB
20/10	Adaptación de S1 y S2, semilla 1	Sin las convoluciones, el modelo cae a 30,05 dB. La adaptación recupera la calidad en pocas épocas

Fecha	Prueba	Qué se obtuvo
18/10	Benchmark v2: costo de las versiones comprimidas	Parámetros, FLOPs, latencia y memoria de todas las versiones
13/10	Benchmark v1: diagnóstico de costo de B0 y S0	El modelo está limitado por memoria y no por cálculo. Se eligió el bloque a reemplazar

Cómo se adaptaron S1 y S2:

- El shared original queda congelado y actúa como profesor.
- Sólo se entrenan los módulos nuevos, 5 épocas de 2.000 pasos.
- La pérdida combina la reconstrucción de la imagen limpia, la imitación de la salida del profesor y la imitación de las salidas de los bloques reemplazados.
- S1 y S2 ven exactamente los mismos datos en el mismo orden.
- La mejor época se elige en validación. El test se usó una sola vez, con la comparación principal fijada de antemano: S2 contra S1.

Reducción de costo

S2 tiene **1,92 M parámetros menos que el baseline, −13,5 %**, y 2,01 M menos que el shared, −14,1 %. Los FLOPs bajan 7,3 % frente al baseline. La latencia casi no cambia, porque el modelo está limitado por los accesos a memoria y no por el cálculo.

Versión	Parámetros	GFLOP a 256×256	Latencia a 256 (ms)	Latencia a 512 (ms)
B0	14,18 M	216,3	41,6	254,0
S0	14,27 M	229,2	43,9	263,2
S0_sel	14,27 M	218,9	42,2	256,3
S1_d32	11,98 M	200,5	41,1	252,8
S2_d32	**12,26 M**	**200,5**	**41,1**	**252,6**

Latencia: mediana en una GPU L4 con batch 1, en float32. FLOPs: conteo del programa antes de optimizar, igual para todas las versiones.

Lo que gana S2 frente al baseline B0:

- **Parámetros:** −13,5 % en total y −16,0 % de parámetros usados por imagen, porque cada imagen usa sólo un experto.
- **Memoria de pesos:** 49 MB en float32, contra 57 MB del baseline.

- **FLOPs:** −7,3 %.
- **Latencia:** −1,1 % a 256×256 y −0,5 % a 512×512.
- **Adaptación barata:** entrenar sólo los módulos nuevos cuesta 49 % menos tiempo por paso y 55 % menos memoria que un paso completo. Son 5,3 GiB contra 11,8 GiB.

Lo que cuestan los expertos frente al módulo común: 277 mil parámetros guardados más, el 2,3 % del modelo. En ejecución S1 y S2 cuestan lo mismo: elegir el experto no agrega tiempo. Calcular los cinco expertos en lugar de elegir uno agregaría entre 1,6 % y 2,4 % de latencia.

Otras palancas medidas, que no dependen de los expertos:

- **bf16:** baja la latencia entre 54 % y 64 %, pero pierde entre 0,67 y 0,90 dB de PSNR macro en validación. Casi toda la pérdida está en dehaze.
- **Rematerialización:** baja 81 % la memoria de entrenamiento, a cambio de 19 % más de tiempo por paso.
- **Dónde está el resto del costo:** los gMLP multi-eje y el cross-gating concentran el 63 % de los parámetros. Sin los gMLP, la latencia bajaría entre 33 % y 40 %.

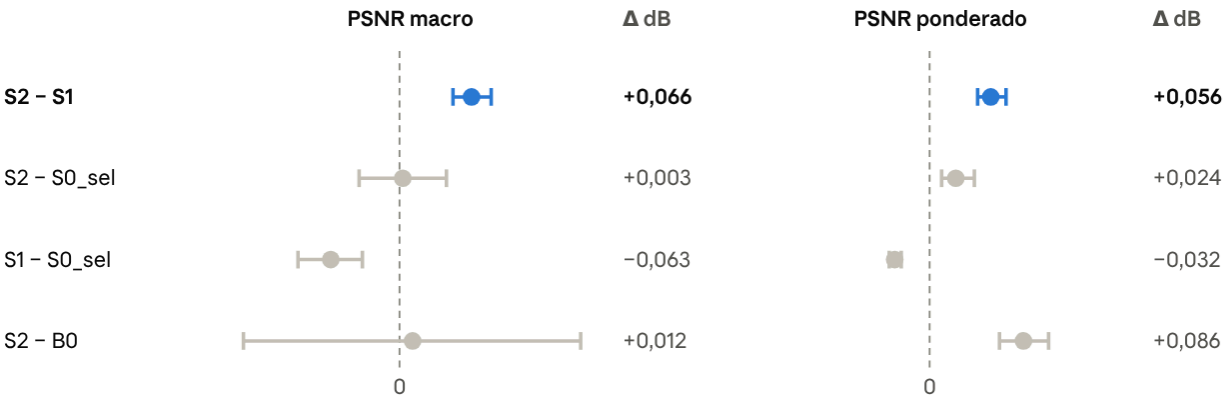
Resultados de calidad

En test, **S2 supera a S1 por 0,066 dB de PSNR macro** y empata con el shared, con un intervalo de confianza que excluye el cero. S1, con el mismo costo que S2, pierde 0,063 dB frente al shared.

Versión	PSNR macro (dB)	PSNR ponderado (dB)	SSIM macro
B0	28,127	27,248	0,8507
S0_sel	28,137	27,309	0,8496
S1_d32, promedio de 2 semillas	28,074	27,278	0,8495
S2_d32, promedio de 2 semillas	**28,140**	**27,334**	**0,8497**

S2 supera a S1 y empata con el shared en test

Diferencia de PSNR en test, en dB, con intervalo de confianza del 95 %. Promedio de las dos semillas.



A la derecha del 0 gana la primera versión. Si el intervalo no toca el 0, la diferencia es clara.

runs_logs/compresion_test_final · 4.730 imágenes de test, bootstrap pareado estratificado por tarea

- **Las dos semillas coinciden:** S2 − S1 da +0,059 dB en una y +0,073 dB en la otra.
- **Por tarea, S2 no pierde frente a S1 en ninguna.** Las ganancias son denoise +0,17 dB, derain +0,08, dehaze +0,06, deblur +0,03 y enhance 0,00.
- **En validación la ventaja de S2 sobre S1 era mayor,** +0,115 dB promediando las dos semillas, IC [+0,086, +0,154]. Es esperable, porque la mejor época se eligió en validación. Allí S2 incluso superaba al shared por 0,055 dB, pero en test empata. En cambio, la pérdida de S1 frente al shared es igual en los dos splits: −0,060 dB en validación y −0,063 en test.
- **El reparto por tarea depende del split, no de la semilla.** En validación, la ventaja de S2 sobre S1 viene sobre todo de dehaze, con +0,33 y +0,41 dB, y en denoise es apenas negativa; las dos semillas dan el mismo patrón. En test, en cambio, la mayor ganancia está en denoise. Lo que se mantiene en las cuatro combinaciones de split y semilla es que S2 supera a S1 en el promedio.
- **Frente al baseline, el intervalo macro es ancho por diferencias que ya tenía el shared.** En enhance pierde 0,37 dB frente al baseline y en dehaze gana entre 0,2 y 0,3 dB. La compresión no las introduce.

Conclusiones y próximo paso

Los expertos por tarea permiten achicar MAXIM un 13,5 % sin perder calidad. Un módulo común del mismo costo no lo logra.

1. **La especialización es lo que conserva la calidad.** S1 y S2 tienen el mismo costo de ejecución, y sólo S2 iguala al shared.
2. **El ahorro es de tamaño y de operaciones, no de tiempo.** Parámetros −13,5 % y FLOPs −7,3 %, con latencia prácticamente igual.

3. **Especializar casi no cuesta:** 2,3 % más de parámetros guardados que el módulo común, y el mismo tiempo de ejecución.
4. **La adaptación es barata:** se entrenan sólo los módulos nuevos, con la mitad de tiempo y memoria por paso.

Próximo paso propuesto: llevar los expertos también a las convoluciones RCAB de los dos niveles menos profundos, de 32 y 64 canales. Ahorraría unos 0,6 M parámetros más, y S2 quedaría cerca de 18 % por debajo del baseline. Usa el mismo código y el mismo protocolo de adaptación, y se decide sólo con validación.

Paso posible después de la opción 1: usar el modelo sin conocer la tarea. Hoy S2 necesita el `task_id` de cada imagen. Calcular los cinco expertos no lo reemplaza: igual hay que elegir uno, y promediarlos aplicaría correcciones de otras degradaciones.

La propuesta es un clasificador liviano del tipo de degradación, entrenado aparte con S2 congelado. Predice la tarea y S2 sigue ejecutando un solo experto. Se evalúa comparando S2 con la tarea predicha contra S2 con la tarea real: la diferencia es lo que cuestan los errores del clasificador. Conviene probarlo también con imágenes de otras fuentes, para descartar que aprenda atajos del dataset.

Comprimir los gMLP multi-eje y el cross-gating daría más ahorro, porque concentran el 63 % de los parámetros, pero es un diseño nuevo y más riesgoso. Queda como trabajo futuro.